# Synthetic Dataset Schemas

**Scope**

This note documents the synthetic datasets stored in:
`data_regular/`, `data_dup/`, `data_def/`, `data_comp/`, `data_rel_mis/`, `data_sur/`.

**How to read**

Each folder has its own schema section -> tables list PK, FK, datatypes, then a short relationship summary.
Non regular folders also list the *intentional data issues* injected (duplicates, orphans, defaults, nulls, broken composites).

---

## data_regular folder (baseline clean retail schema)

**Purpose**

Clean baseline schema used for normal relationship discovery and scoring.

### customers

**PK:** `CustomerID`
**Unique:** `Email`

| Column | Type |
|---|---|
| CustomerID | int |
| Email | string |
| FirstName | string |
| LastName | string |
| SignupDate | datetime |
| IsActive | bool |

### suppliers

**PK:** `SupplierID`
**Unique:** `SupplierCode`

| Column | Type |
|---|---|
| SupplierID | int |
| SupplierCode | string |
| SupplierName | string |
| Country | string |

### categories

**PK:** `CategoryID`
**Unique:** `CategoryCode`

| Column | Type |
|---|---|
| CategoryID | int |
| CategoryCode | string |
| CategoryName | string |

## products

**PK:** `ProductID`
**Unique:** `SKU`
**FK:** `CategoryID -> categories.CategoryID`
**FK:** `SupplierID -> suppliers.SupplierID`

| Column | Type |
|---|---|
| ProductID | int |
| SKU | string |
| ProductName | string |
| CategoryID | int |
| SupplierID | int |
| UnitPrice | float |
| IsDiscontinued | bool |

## orders

**PK:** `OrderID`
**Unique:** `OrderNumber`
**FK:** `CustomerID -> customers.CustomerID`

| Column | Type |
|---|---|
| OrderID | int |
| OrderNumber | string |
| CustomerID | int |
| OrderDate | datetime |
| Currency | string |
| OrderStatus | string |

## order_lines

**PK:** `OrderLineID`
**FK:** `OrderID -> orders.OrderID`
**FK:** `ProductID -> products.ProductID`

| Column | Type |
|---|---|
| OrderLineID | int |
| OrderID | int |
| ProductID | int |
| Quantity | int |
| UnitPriceAtSale | float |
| DiscountPct | float |
| LineTotal | float |

## payments

**PK:** `PaymentID`
**FK:** `OrderID -> orders.OrderID`
**Cardinality:** 0..many per order

| Column | Type |
|---|---|
| PaymentID | int |
| OrderID | int |
| TxnRef | string |

| Column | Type |
|---|---|
| PaymentMethod | string |
| PaidAt | datetime |
| Amount | float |

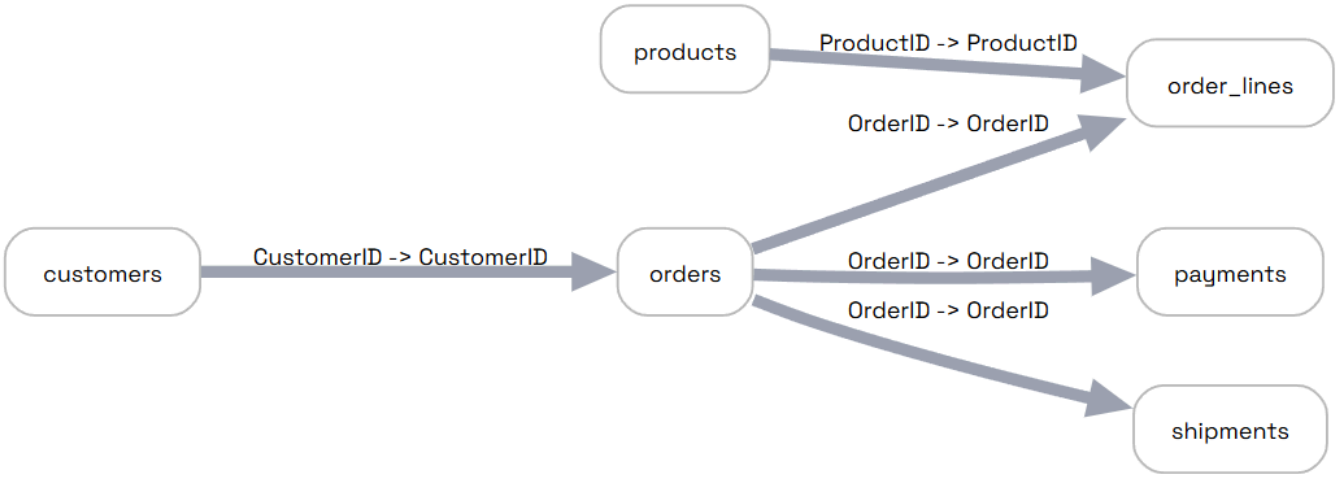
### shipments

**PK:** `ShipmentID`
**FK:** `OrderID -> orders.OrderID`
**Cardinality:** 0..many per order

| Column | Type |
|---|---|
| ShipmentID | int |
| OrderID | int |
| TrackingNumber | string |
| Carrier | string |
| ShippedAt | datetime |

#### Relationship summary

- customers 1 -> many orders
- orders 1 -> many order_lines
- products 1 -> many order_lines
- categories 1 -> many products
- suppliers 1 -> many products
- orders 1 -> 0..many payments
- orders 1 -> 0..many shipments




---

## data_dup folder (duplication + entity resolution noise)

⚠ **Intentional issues**

- duplicated cell values (emails)
- exact duplicate rows
- same real person can appear under multiple IDs across systems (customers vs crm_customers)
- bridge table can be many to many

### customers

**PK (intended):** `customer_id`
**Issues:** duplicated `email`, duplicate rows

| Column | Type |
|---|---|
| customer_id | int64 |
| email | string |
| full_name | string |
| created_at | datetime |

## orders

**PK:** `order_id`
**FK:** `customer_id -> customers.customer_id`

| Column | Type |
|---|---|
| order_id | int64 |
| customer_id | int64 |
| order_total | float |
| order_date | datetime |

## crm_customers

**PK:** `crm_customer_id`
**Dirty join key:** `email` overlaps with `customers.email`
**Issues:** same `email` mapped to multiple CRM IDs

| Column | Type |
|---|---|
| crm_customer_id | int64 |
| email | string |
| phone | string |

## payments

**PK (intended):** `payment_id`
**FK:** `order_id -> orders.order_id`
**Issues:** duplicate rows appended

| Column | Type |
|---|---|
| payment_id | int |
| order_id | int |
| amount | float |
| status | string |
| method | string |

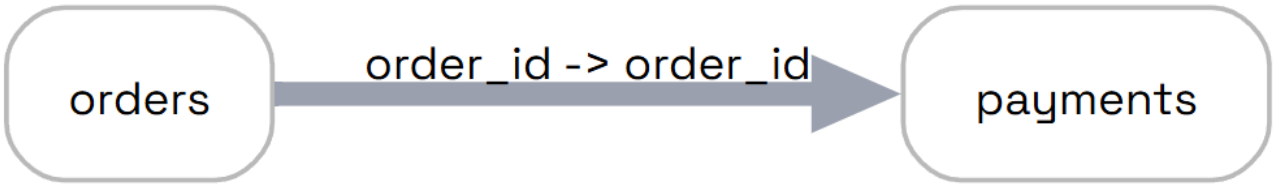
## customer_identity_map

Bridge built via `email` join ( `customers` LEFT JOIN `crm_customers` )
**Issues:** one `customer_id` can map to multiple `crm_customer_id`

| Column | Type |
|---|---|
| customer_id | int64 |
| email | string |
| crm_customer_id | float or int (nullable) |

**Relationship summary**

- customers 1 -> many orders
- orders 1 -> 0..many payments (plus duplicates)
- customers <-> crm_customers linked by email (not 1 to 1)
- identity_map is noisy many to many style mapping




## data_def folder (default FK + sentinel values)

⚠ **Intentional issues**

- dominant default FK values (0, -1, "unknown")
- orphan sentinel values (9999, 888888, "NA_USER")
- includes dummy parent rows to represent defaults

### customers

**PK:** `customer_id` (includes dummy `0`)

| Column | Type |
|---|---|
| customer_id | int |
| customer_name | string |
| is_dummy | bool |

### orders

**PK:** `order_id`
**FK candidate:** `customer_id -> customers.customer_id`
**Issues:** frequent `0`, orphan `9999`, popular real `42`

| Column | Type |
|---|---|
| order_id | int |
| customer_id | int |
| order_amount | float |
| status_code | int |

### products

**PK:** `product_id` (includes dummy `-1`)

| Column | Type |
|---|---|
| product_id | int |
| product_name | string |
| is_dummy | bool |

### order_items

**PK:** `item_id`
**FK:** `order_id -> orders.order_id`

**FK candidate:** `product_id -> products.product_id`
**Issues:** frequent `-1`, orphan `888888`, popular real `7`

| Column | Type |
|---|---|
| item_id | int |
| order_id | int |
| product_id | int |
| qty | int |
| unit_price | float |

### users

**PK:** `user_key` (includes dummy `"unknown"`)

| Column | Type |
|---|---|
| user_key | string |
| email | string |
| is_dummy | bool |

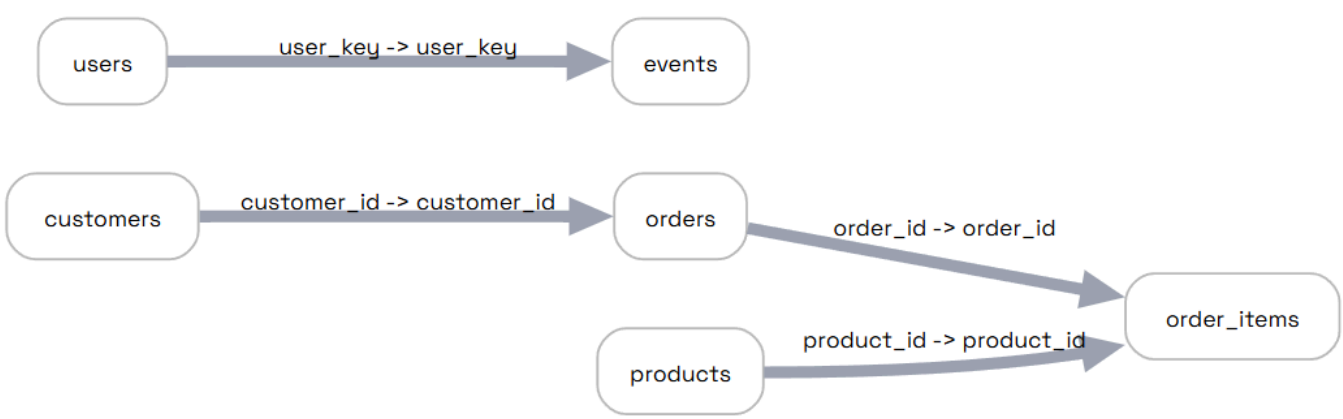
### events

**PK:** `event_id`
**FK candidate:** `user_key -> users.user_key`
**Issues:** frequent `"unknown"`, orphan `"NA_USER"`

| Column | Type |
|---|---|
| event_id | int |
| user_key | string |
| event_type | string |

#### Relationship summary

- customers 1 -> many orders (default 0 + orphan 9999 present)
- products 1 -> many order_items (default -1 + orphan 888888 present)
- users 1 -> many events (default "unknown" + orphan "NA_USER" present)




## data_comp folder (composite UCC test cases)

⚠ **Intentional issues**

- tables where unary keys are deliberately non unique to surface composite keys
- one valid composite UCC and two broken composite candidates (duplicates, nulls)

## customers

**PK:** `customer_id` (includes dummy `0`)

| Column | Type |
|---|---|
| customer_id | int |
| customer_name | string |
| is_dummy | bool |

## orders

`order_id` is intentionally non unique
**Composite UCC (valid):** ( `customer_id` , `order_number` )
**Issues:** default `0` frequent, orphan `9999`

| Column | Type |
|---|---|
| order_id | int |
| customer_id | int |
| order_number | int |
| order_amount | float |
| status_code | int |

## products

**PK:** `product_id` (includes dummy `-1`)

| Column | Type |
|---|---|
| product_id | int |
| product_name | string |
| is_dummy | bool |

## order_items

**PK:** `item_id`
**Composite key candidate (broken):** ( `order_id` , `line_no` )
**Issues:** 2 percent duplicates injected into ( `order_id` , `line_no` ), default `-1` , orphan `888888`

| Column | Type |
|---|---|
| item_id | int |
| order_id | int |
| line_no | int |
| product_id | int |
| qty | int |
| unit_price | float |

## users

**PK:** `user_key` (includes dummy `"unknown"`)

| Column | Type |
|---|---|
| user_key | string |
| email | string |
| is_dummy | bool |

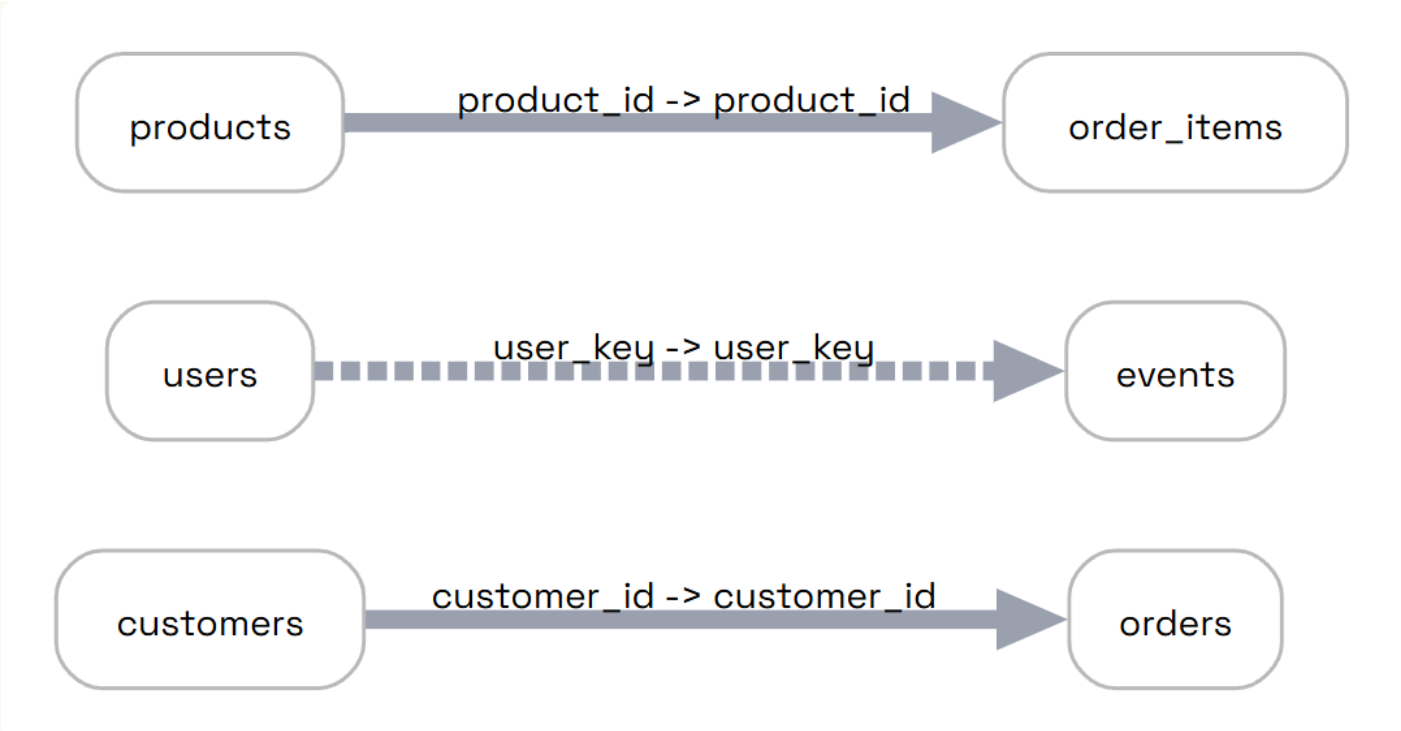
## events

`event_id` is intentionally non unique
**Composite key candidate (broken by nulls):** ( `user_key` , `event_ts` )
**Issues:** default `"unknown"` , orphan `"NA_USER"` , nulls in `event_ts` and `user_key`

| Column | Type |
|---|---|
| event_id | int |
| user_key | string (nullable) |
| event_ts | datetime (nullable) |
| event_type | string |

**Relationship summary**

- orders has valid composite UCC ( `customer_id` , `order_number` )
- order_items breaks composite uniqueness via duplicates
- events breaks composite uniqueness via nulls




## data_rel_mis folder (relational missingness + dangling FKs)

⚠ **Intentional issues**

- orphan FK values in parent child direction
- null FK values
- missing child rows (orders with no payments)
- dangling child rows (payments with unknown order_id)
- optional duplicates or split payments

### customers

**PK:** `customer_id`

| Column | Type |
|---|---|
| customer_id | int64 |
| full_name | string |
| email | string |
| created_at | datetime |

## orders

**PK:** `order_id`
**FK candidate:** `customer_id -> customers.customer_id`
**Issues:** orphan `999999`, null `customer_id` (stored as `Int64`)

| Column | Type |
|---|---|
| order_id | int64 |
| customer_id | Int64 (nullable) |
| order_date | datetime |
| order_total | float |

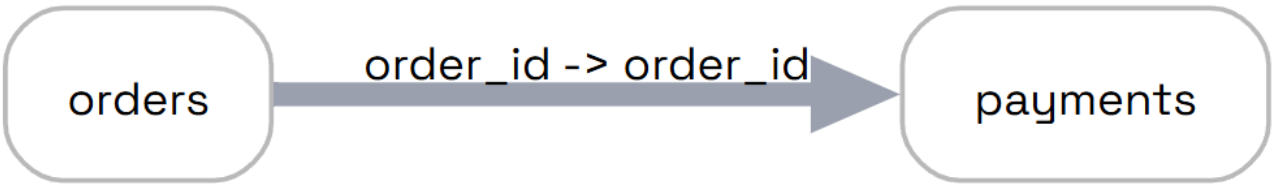
## payments

**PK:** `payment_id`
**FK candidate:** `order_id -> orders.order_id`
**Issues:** some orders have 0 payments, some payments reference `888888`, some orders have multiple payments

| Column | Type |
|---|---|
| payment_id | int |
| order_id | int |
| paid_at | datetime |
| amount | float |
| method | string |
| status | string |

#### Relationship summary

- customers 1 -> many orders, but some orders have null or orphan customer_id
- orders 1 -> 0..many payments, with deliberate missing and dangling cases




## data_sur folder (surrogate keys + mixed identifiers)

⚠ **Intentional issues**

- multiple parallel identifiers for the same entity (int PK + UUID)
- orphan and null FKs in both identifier forms
- sequential invoice numbers with small gaps (surrogate like behaviour)

## customers

**PK candidates:** `customer_pk_int`, `customer_uuid`
**Unique candidates:** `email`, `customer_code`

| Column | Type |
|---|---|
| customer_pk_int | int64 |
| customer_uuid | string |
| email | string |

| Column | Type |
|---|---|
| full_name | string |
| customer_code | string |
| created_at | datetime |

## products

**PK:** `product_id`
**Unique candidates:** `sku`, `barcode`

| Column | Type |
|---|---|
| product_id | int64 |
| sku | string |
| barcode | string |
| product_name | string |
| price | float |

## promotions

**PK:** `promo_code`

| Column | Type |
|---|---|
| promo_code | string |
| description | string |
| discount_pct | int |

## orders

**PK candidates:** `order_id`, `order_uuid`
**FK candidates:**

- `customer_pk_int -> customers.customer_pk_int`
- `customer_uuid -> customers.customer_uuid`
- `promo_code -> promotions.promo_code` (nullable)
  **Issues:** orphan + null customer refs, invoice_number has gaps

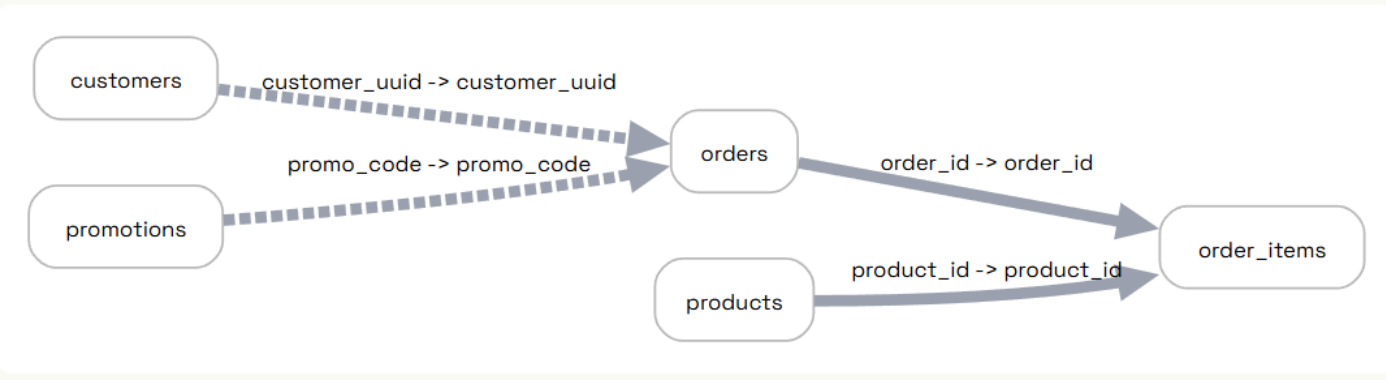
| Column | Type |
|---|---|
| order_id | int64 |
| order_uuid | string |
| order_date | datetime |
| order_total | float |
| customer_pk_int | Int64 (nullable) |
| customer_uuid | string (nullable) |
| invoice_number | int64 |
| promo_code | string (nullable) |

## order_items

**PK candidates:** `order_item_id`, `line_uuid`
**FK:** `order_id -> orders.order_id`
**FK:** `product_id -> products.product_id`

| Column | Type |
|---|---|
| order_item_id | int64 |
| line_uuid | string |

| Column | Type |
|---|---|
| order_id | int64 |
| product_id | int64 |
| qty | int64 |
| unit_price | float |

**Relationship summary**

- customers 1 -> many orders via both int and uuid identifiers (with deliberate orphans + nulls)
- promotions 1 -> many orders (sparse)
- orders 1 -> many order_items
- products 1 -> many order_items